

COMP0497 – Algoritmos e Estrutura de Dados 1

Análise de Complexidade: Algoritmos Iterativos e Recursivos

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Parte I – Fundamentos

- Por que analisar o pior caso?
- Regras para contar operações

Parte II – Algoritmos Iterativos

- De $O(1)$ até $O(n^3)$: exemplos progressivos
- Padrões para reconhecer complexidades

Parte III – Algoritmos Recursivos

- Relações de recorrência
- 10 padrões recursivos clássicos

Parte IV – Resolvendo Recorrências

- Árvore de recorrência
- Método da substituição
- Teorema Mestre + exemplos

Parte I – Fundamentos

Ao final desta aula, você deverá ser capaz de:

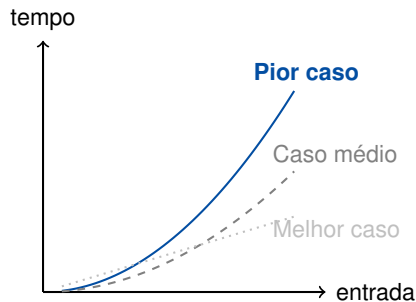
1. Justificar por que a **análise de pior caso** é o padrão da área.
2. Aplicar as regras de contagem para determinar a complexidade de **algoritmos iterativos**.
3. Formular **relações de recorrência** para algoritmos recursivos.
4. Resolver recorrências usando **árvore**, **substituição** e **Teorema Mestre**.

Analogia: Projeto Estrutural

Um engenheiro civil não projeta uma ponte para suportar o dia típico de tráfego — ele projeta para o cenário mais extremo.

Na prática, isso nos dá:

- **Garantia:** O algoritmo *nunca* será mais lento que o limite.
- **Segurança:** Essencial em sistemas de tempo real.
- **Comparabilidade:** Um critério justo para comparar algoritmos distintos.



Regra de Ouro

Durante a análise, assumimos que **laços de repetição percorrem o máximo de iterações** e que **decisões seguem o caminho de maior custo**.

A notação Big-O (O) captura esse **limite superior assintótico**: ignoramos constantes e termos de menor ordem, focando apenas no comportamento dominante quando $n \rightarrow \infty$.

Construção	Regra de análise
Operação básica	Atribuições, aritméticas e acessos a array custam $O(1)$.
Sequência	O custo total é o do comando mais caro (Regra da Soma).
Decisão (if/else)	No pior caso, assumimos o ramo de maior custo.
Laço (for/while)	Custo do corpo $\times$ número máximo de iterações (Regra do Produto).
Laços consecutivos	Somam-se as complexidades (Regra da Soma).
Laços aninhados	Multiplicam-se as complexidades (Regra do Produto).

Parte II – Algoritmos Iterativos

Padrão: Acesso Direto

Operações que não dependem do tamanho da entrada.

```
READ-SENSOR(A, i)
1  if i <= A.length
2      return A[i]
3  return NIL
```

Análise: Nenhum laço, nenhuma chamada recursiva. O tempo é fixo quer o arranjo tenha 10 ou 10.000 elementos.

Complexidade: $O(1)$

Padrão: Um laço que percorre todos os n elementos.

```
CALCULATE-RESULTANT-FORCE(F)
```

```
1  total = 0
```

```
2  for i = 1 to F.length
```

```
3      total = total + F[i]
```

```
4  return total
```

Análise: O corpo do laço custa $O(1)$ e executa n vezes.

Complexidade: $O(n)$

Padrão: Retorno antecipado, mas pior caso percorre tudo.

```
CHECK-MATERIAL-FAILURE(T, yield_limit)
1  for i = 1 to T.length
2      if T[i] > yield_limit
3          return TRUE
4  return FALSE
```

Análise: No melhor caso, encontra na primeira posição ($O(1)$). No **pior caso**, o elemento não está na lista — percorre todos os n elementos.

Complexidade (pior caso): $O(n)$

Padrão: Dois laços independentes em sequência (Regra da Soma).

NORMALIZE-TOPOGRAPHIC-DATA(C)

```
1  c_max = C[1]
2  for i = 2 to C.length           // primeiro laço:  $O(n)$ 
3      if C[i] > c_max
4          c_max = C[i]
5  let N be a new array of size C.length
6  for i = 1 to C.length           // segundo laço:  $O(n)$ 
7      N[i] = C[i] / c_max
8  return N
```

Análise: $O(n) + O(n) = O(2n)$. Pela Regra da Soma, a constante é ignorada.

Complexidade: $O(n)$

Juntando tudo: operação básica + laço + decisão

```
def encontrar_tensao_maxima(leituras):  
    maximo = leituras[0]           #  $O(1)$   
    for tensao in leituras:        # Executa  $n$  vezes  
        if tensao > maximo:        #  $O(1)$   
            maximo = tensao        #  $O(1)$   
    return maximo                  #  $O(1)$ 
```

- Operações dentro do laço: $O(1)$.
- Laço executa n vezes: Regra do Produto $\Rightarrow O(n) \cdot O(1) = O(n)$.
- Operações fora do laço: $O(1)$. Regra da Soma $\Rightarrow O(n) + O(1) = O(n)$.

Complexidade: $O(n)$

Padrão: Dois laços aninhados, cada um percorrendo n elementos.

CALCULATE-POINT-DISTANCES(P)

```
1  n = P.length
2  for i = 1 to n
3      for j = 1 to n
4          dx = P[i].x - P[j].x
5          dy = P[i].y - P[j].y
6          dist = sqrt(dx^2 + dy^2)
7          APPEND(D, dist)
8  return D
```

Análise: Laço externo: n vezes. Laço interno: n vezes. Regra do Produto:
 $O(n) \cdot O(n)$.

Complexidade: $O(n^2)$

Padrão: O laço interno depende do índice do externo (comum em ordenações).

SELECTION-SORT-LOADS(L)

```
1  n = L.length
2  for i = 1 to n - 1
3      menor = i
4      for j = i + 1 to n
5          if L[j] < L[menor]
6              menor = j
7      exchange L[i] with L[menor]
```

Análise: iterações do laço interno: $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$ (progressão aritmética).

Complexidade: $O(n^2)$

Padrão: Dois tamanhos distintos de entrada geram duas variáveis na complexidade.

PROCESS-THERMAL-MESH(M)

```
1  linhas = M.rows
2  colunas = M.columns
3  calor_total = 0
4  for i = 1 to linhas
5      for j = 1 to colunas
6          calor_total = calor_total + M[i, j]
7  return calor_total
```

Análise: Laço externo: n vezes. Laço interno: m vezes. Não podemos simplificar para n^2 porque $n \neq m$ em geral.

Complexidade: $O(n \cdot m)$

Padrão: A cada iteração, o espaço de busca é dividido por um fator constante.

```
BINARY-SEARCH-STRESS(T, alvo)
1  inicio = 1
2  fim = T.length
3  while inicio <= fim
4      meio = floor((inicio + fim) / 2)
5      if T[meio] == alvo
6          return meio
7      elseif T[meio] < alvo
8          inicio = meio + 1
9      else fim = meio - 1
10 return NIL
```

Análise: O espaço é dividido por 2 a cada iteração: $n, n/2, n/4, \dots, 1$. Isso leva $\log_2 n$ passos.

Complexidade: $O(\log n)$

Padrão: Para cada um dos n elementos, executamos uma operação $O(\log n)$.

```
PROCESS-BATCH-SEARCHES(T_ordenado, consultas)
1  n = consultas.length
2  let R be a new array of size n
3  for i = 1 to n
4      idx = BINARY-SEARCH-STRESS(T_ordenado, consultas[i])
5      R[i] = idx
6  return R
```

Análise: Laço externo: n vezes. Corpo (busca binária): $O(\log m)$. Regra do Produto: $O(n \log m)$.

Se m e n crescem proporcionalmente, escrevemos $O(n \log n)$.

Complexidade: $O(n \log n)$

Padrão: Três laços aninhados (comum em operações matriciais).

```
MULTIPLY-STIFFNESS-MATRICES(A, B)
```

```
1  n = A.rows
2  let C be a new n x n matrix initialized with 0s
3  for i = 1 to n
4      for j = 1 to n
5          for k = 1 to n
6              C[i,j] = C[i,j] + A[i,k] * B[k,j]
7  return C
```

Análise: Três laços de n iterações cada: $O(n) \cdot O(n) \cdot O(n)$.

Complexidade: $O(n^3)$

Complexidade	Padrão típico	Exemplo
$O(1)$	Acesso direto, sem laço	Leitura de sensor
$O(\log n)$	Divide espaço por constante	Busca binária
$O(n)$	Um laço sobre n elementos	Soma de vetores
$O(n \log n)$	Laço $\times$ operação $\log$	n buscas binárias
$O(n^2)$	Dois laços aninhados	Distâncias entre pontos
$O(n^3)$	Três laços aninhados	Multiplicação de matrizes

Dica Prática

Conte o número de laços aninhados *independentes* que dependem de n : 1 laço $\approx O(n)$, 2 laços $\approx O(n^2)$, 3 laços $\approx O(n^3)$. Se o espaço é dividido a cada passo, pense em $\log n$.

Parte III – Algoritmos Recursivos

Iterativo — já sabemos analisar:

- Contamos laços explícitos.
- Aplicamos Regra da Soma e do Produto.

Recursivo — nova abordagem:

- Não há laços explícitos para contar.
- O algoritmo chama *a si mesmo*.
- Precisamos de **Relações de Recorrência**.

Ideia Central

Expressamos o tempo $T(n)$ em função do tempo para resolver instâncias menores do mesmo problema: $T(\text{algo menor que } n)$.

Uma relação de recorrência tem duas partes:

1. **Caso base:** Quando a entrada é suficientemente pequena, o custo é constante. Ex: $T(1) = \Theta(1)$.
2. **Passo recursivo:** O custo para a entrada n depende de chamadas menores. Ex: $T(n) = 2T(n/2) + \Theta(n)$.

Duas formas mais comuns:

Forma	Significado
$T(n) = aT(n/b) + f(n)$	Divide a entrada por fator b , gera a subproblemas
$T(n) = T(n-1) + f(n)$	Reduz a entrada em 1 unidade

10 Padrões Recursivos Clássicos

Agrupados por tipo de redução da entrada:

Grupo	Padrões
Decréscimo ($n - 1$)	1. Fatorial, 2. Fibonacci, 6. Quick Sort (pior) 7. Hanói, 8. Potência
Divisão (n/b)	3. Busca binária, 4. Máximo por divisão, 5. Merge Sort 9. Exponenciação rápida, 10. Mult. de matrizes


```
FACTORIAL(n)
1  if n == 0
2      return 1
3  return n * FACTORIAL(n - 1)
```

Recorrência:

$$T(n) = T(n - 1) + \Theta(1)$$

Intuição: Cada chamada faz $O(1)$ de trabalho e gera *uma* chamada com entrada $n-1$. A profundidade é n .

Complexidade: $O(n)$

FIBONACCI(n)

```
1  if n == 0 or n == 1
2      return n
3  return FIBONACCI(n - 1)
4      + FIBONACCI(n - 2)
```

Recorrência:

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

Intuição: Cada chamada gera *duas* novas chamadas. A árvore de recursão praticamente **dobra** a cada nível.

Contraste com Fatorial

Fatorial: 1 chamada por nível $\Rightarrow O(n)$. Fibonacci: ~ 2 chamadas por nível $\Rightarrow O(2^n)$. A diferença entre linear e exponencial está no **número de ramificações**.

Complexidade: $O(2^n)$

```
RECURSIVE-BINARY-SEARCH(A, p, r, x)
1  if p > r
2      return NIL
3  q = floor((p + r) / 2)
4  if A[q] == x
5      return q
6  elseif A[q] > x
7      return REC-BIN-SEARCH(A, p, q-1, x)
8  else
9      return REC-BIN-SEARCH(A, q+1, r, x)
```

Recorrência:

$$T(n) = T(n/2) + \Theta(1)$$

Intuição: Divide pela metade, mas explora apenas **um** lado.

Profundidade: $\log n$.

Complexidade: $O(\log n)$

```
FIND-MAXIMUM(A, p, r)
1  if p == r
2      return A[p]
3  q = floor((p + r) / 2)
4  max_esq = FIND-MAXIMUM(A, p, q)
5  max_dir = FIND-MAXIMUM(A, q + 1, r)
6  if max_esq > max_dir return max_esq
7  else return max_dir
```

Recorrência: $T(n) = 2T(n/2) + \Theta(1)$

Diferença-chave em relação à Busca Binária: Aqui exploramos **ambos** os lados ($a = 2$ ao invés de $a = 1$). O custo é dominado pelas folhas da árvore.

Complexidade: $O(n)$

Padrão 5: Merge Sort — Divisão Balanceada + Combinação



```
MERGE-SORT(A, p, r)
1  if p < r
2      q = floor((p + r) / 2)
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)          // Custo Theta(n)
```

Recorrência: $T(n) = 2T(n/2) + \Theta(n)$

Comparação: Padrão 4 vs. 5

	FIND-MAX	MERGE-SORT
a (subproblemas)	2	2
n/b (divisão)	$n/2$	$n/2$
$f(n)$ (combinar)	$\Theta(1)$	$\Theta(n)$
Resultado	$O(n)$	$O(n \log n)$

O custo de **combinar** faz toda a diferença!

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)      // Custo Theta(n)
3      QUICKSORT(A, p, q - 1)
4      QUICKSORT(A, q + 1, r)
```

Recorrência (pior caso): $T(n) = T(n - 1) + \Theta(n)$

Intuição: No pior caso (arranjo já ordenado), a partição gera um subproblema de tamanho $n-1$ e outro vazio. A árvore degenera em uma lista:

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n(n+1)}{2}$$

Complexidade (pior caso): $O(n^2)$

```
HANOI-TOWER(n, origem, destino, auxiliar)
1  if n == 1
2      print "Move disco da origem para o destino"
3  else
4      HANOI-TOWER(n - 1, origem, auxiliar, destino)
5      print "Move disco da origem para o destino"
6      HANOI-TOWER(n - 1, auxiliar, destino, origem)
```

Recorrência: $T(n) = 2T(n - 1) + \Theta(1)$

Contraste com Fibonacci: Mesma ramificação ($a = 2$), mas aqui a redução é por 1 (ao invés de $n/2$). Expandindo: $T(n) = 2^n - 1$, uma progressão geométrica.

Complexidade: $O(2^n)$

```
POWER(a, n)
1  if n == 0
2      return 1
3  return a * POWER(a, n - 1)
```

Recorrência:

$$T(n) = T(n - 1) + \Theta(1)$$

Idêntica ao Fatorial.

$$O(n)$$

Padrão 9: Exponenciação Rápida

```
FAST-POWER(a, n)
1  if n == 0 return 1
2  if n % 2 == 0
3      temp = FAST-POWER(a, n/2)
4      return temp * temp
5  else
6      return a * FAST-POWER(a, n-1)
```

Recorrência:

$$T(n) = T(n/2) + \Theta(1)$$

$$O(\log n)$$

Lição

A diferença entre $O(n)$ e $O(\log n)$ está em **como** a entrada é reduzida: subtraindo 1 vs. dividindo por 2.


```
RECURSIVE-MATRIX-MULTIPLY(A, B)
1  n = A.rows
2  if n == 1 return A[1,1] * B[1,1]
3  let C be a new n x n matrix
4  Partition A, B, C into n/2 x n/2 submatrices
5  C11 = ADD(MULT(A11, B11), MULT(A12, B21))
6  // ... repete para C12, C21, C22 (8 multiplicações)
7  return C
```

Recorrência: $T(n) = 8 T(n/2) + \Theta(n^2)$

Análise: Pelo Teorema Mestre, $n^{\log_2 8} = n^3$ domina n^2 (Caso 1).

Complexidade: $O(n^3)$

(O algoritmo de Strassen reduz de 8 para 7 multiplicações: $O(n^{\log_2 7}) \approx O(n^{2.81})$.)

#	Algoritmo	Recorrência	Complexidade
1	Fatorial	$T(n-1) + \Theta(1)$	$O(n)$
2	Fibonacci ingênuo	$T(n-1) + T(n-2) + \Theta(1)$	$O(2^n)$
3	Busca binária	$T(n/2) + \Theta(1)$	$O(\log n)$
4	Máximo por divisão	$2T(n/2) + \Theta(1)$	$O(n)$
5	Merge Sort	$2T(n/2) + \Theta(n)$	$O(n \log n)$
6	Quick Sort (pior)	$T(n-1) + \Theta(n)$	$O(n^2)$
7	Hanói	$2T(n-1) + \Theta(1)$	$O(2^n)$
8	Potência ingênua	$T(n-1) + \Theta(1)$	$O(n)$
9	Exponenciação rápida	$T(n/2) + \Theta(1)$	$O(\log n)$
10	Mult. matrizes (ingênua)	$8T(n/2) + \Theta(n^2)$	$O(n^3)$

Antes de prosseguir, observe os padrões na tabela anterior:

1. $T(n-1) + \Theta(1)$ sempre dá $O(n)$. **Por quê?** Porque são n chamadas, cada uma custando $\Theta(1)$.
2. $T(n/2) + \Theta(1)$ sempre dá $O(\log n)$. **Por quê?** Porque dividir n por 2 repetidamente leva $\log n$ passos.
3. O que diferencia $O(n)$ de $O(n \log n)$ em padrões com $2T(n/2)$? O custo de **combinar**: $\Theta(1)$ vs. $\Theta(n)$.
4. O que torna $2T(n-1)$ exponencial e $T(n-1)$ linear? O **número de ramificações**: 2 chamadas vs. 1 chamada.

Parte IV – Resolvendo Recorrências

Uma recorrência como $T(n) = 2T(n/2) + \Theta(n)$ **não nos dá** a complexidade assintótica diretamente. Precisamos *resolvê-la*.

Método	Para que serve	Força
Árvore	Expandir a recorrência visualmente para gerar um palpite	Intuição
Substituição	Provar formalmente (por indução) que o palpite está correto	Rigor
Teorema Mestre	“Receita de bolo” para $T(n) = aT(n/b) + f(n)$	Rapidez

Fluxo típico: Árvore (gerar palpite) $\rightarrow$ Substituição (provar) ou Teorema Mestre (quando aplicável).

Método 1: Árvore de Recorrência

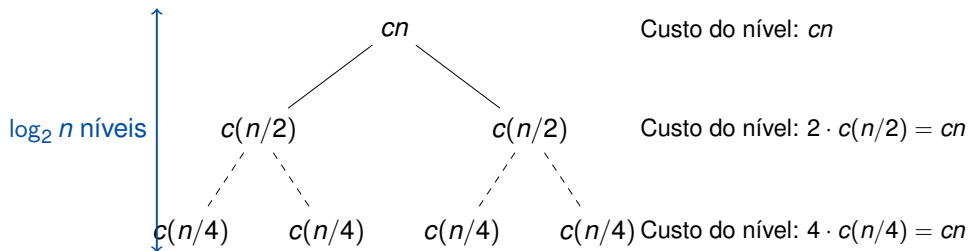
Procedimento:

1. Desenhe a árvore: cada nó mostra o custo não-recursivo daquela chamada.
2. Some os custos de todos os nós **em cada nível**.
3. Some os custos de **todos os níveis** para obter o total.

Aplicaremos ao exemplo clássico

$$T(n) = 2T(n/2) + cn$$

Árvore: $T(n) = 2T(n/2) + cn$



Cada nível custa exatamente cn . São $\log_2 n$ níveis.

Total: $cn \cdot \log_2 n = \Theta(n \log n)$.

Dois passos:

1. **Adivinhar** a forma da solução (geralmente via árvore).
2. **Provar por indução** que a solução funciona.

Exemplo: $T(n) = 2T(\lfloor n/2 \rfloor) + n$

Palpite (obtido da árvore): $T(n) = O(n \log n)$.

Devemos provar que $T(n) \leq c n \log n$ para algum $c > 0$.

Hipótese Indutiva: $T(k) \leq c k \log k$ para todo $k < n$.

Passo Indutivo:

$$\begin{aligned} T(n) &= 2 T(\lfloor n/2 \rfloor) + n \\ &\leq 2 c (n/2) \log(n/2) + n \\ &= c n (\log n - \log 2) + n \\ &= c n \log n - c n + n \\ &\leq c n \log n \quad \text{desde que } c \geq 1. \quad \square \end{aligned}$$

Método 3: Teorema Mestre

Aplica-se a recorrências da forma padrão

$$T(n) = a T(n/b) + f(n) \quad (a \geq 1, b > 1)$$

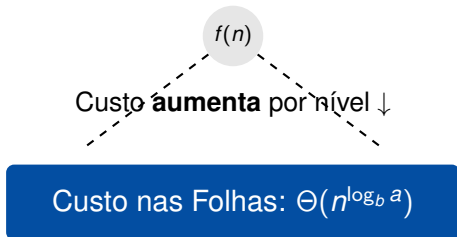
A essência: comparamos o custo de dividir/combinar $f(n)$ com o “peso das folhas” $n^{\log_b a}$.

Quem domina?

Caso	Condição	Resultado
1 – Folhas dominam	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$
2 – Empate	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
3 – Raiz domina	$f(n) = \Omega(n^{\log_b a + \epsilon})$ + regularidade	$T(n) = \Theta(f(n))$

Caso 1: As Folhas Dominam

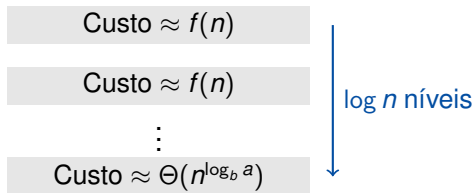
$$f(n) = O(n^{\log_b a - \epsilon}) \text{ para algum } \epsilon > 0 \quad \Rightarrow \quad T(n) = \Theta(n^{\log_b a})$$



Interpretação: A maior parte do trabalho está na base da árvore. O custo de dividir/combinar é insignificante.

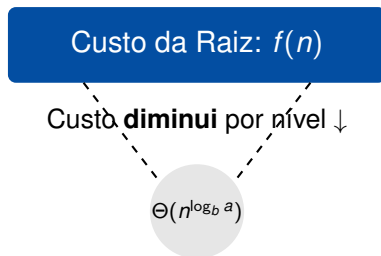
Caso 2: Empate

$$f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$$



Interpretação: Trabalho uniforme em todos os níveis. Total = custo de um nível $\times$ número de níveis.

$$f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ (+ condição de regularidade)} \Rightarrow T(n) = \Theta(f(n))$$



Interpretação: O custo de dividir/combinar domina completamente os subproblemas.

Teorema Mestre: 10 Exemplos Resolvidos

Em cada exemplo, identificaremos a , b , $f(n)$ e $n^{\log_b a}$, compararemos, e aplicaremos o caso correto.

Exemplo 1: $T(n) = 4T(n/2) + n$ — Caso 1



1. **Parâmetros:** $a = 4$, $b = 2$, $f(n) = n$.
2. **Folhas:** $n^{\log_2 4} = n^2$.
3. **Comparação:** $f(n) = n = O(n^{2-1})$. As folhas dominam.

$$T(n) = \Theta(n^2)$$

Exemplo 2: $T(n) = 2T(n/2) + n$ — Caso 2 (Merge Sort)



1. **Parâmetros:** $a = 2$, $b = 2$, $f(n) = n$.
2. **Folhas:** $n^{\log_2 2} = n$.
3. **Comparação:** $f(n) = n = \Theta(n)$. Empate.

$$T(n) = \Theta(n \log n)$$

Exemplo 3: $T(n) = T(n/2) + n$ — Caso 3

1. **Parâmetros:** $a = 1$, $b = 2$, $f(n) = n$.
2. **Folhas:** $n^{\log_2 1} = n^0 = 1$.
3. **Comparação:** $f(n) = n = \Omega(n^{0+1})$. A raiz domina.
4. **Regularidade:** $1 \cdot (n/2) = n/2 \leq cn$ para $c = 1/2 < 1$. ✓

$$T(n) = \Theta(n)$$

Exemplo 4: $T(n) = 8T(n/2) + n^2$ — Caso 1

1. **Parâmetros:** $a = 8$, $b = 2$, $f(n) = n^2$.
2. **Folhas:** $n^{\log_2 8} = n^3$.
3. **Comparação:** $f(n) = n^2 = O(n^{3-1})$. As folhas dominam.

$$T(n) = \Theta(n^3)$$

Exemplo 5: $T(n) = 4T(n/2) + n^2$ — Caso 2

1. **Parâmetros:** $a = 4$, $b = 2$, $f(n) = n^2$.
2. **Folhas:** $n^{\log_2 4} = n^2$.
3. **Comparação:** $f(n) = n^2 = \Theta(n^2)$. Empate.

$$T(n) = \Theta(n^2 \log n)$$

Exemplo 6: $T(n) = 2T(n/2) + n^2$ — Caso 3

1. **Parâmetros:** $a = 2$, $b = 2$, $f(n) = n^2$.
2. **Folhas:** $n^{\log_2 2} = n$.
3. **Comparação:** $f(n) = n^2 = \Omega(n^{1+1})$. A raiz domina.
4. **Regularidade:** $2(n/2)^2 = n^2/2 \leq cn^2$ para $c = 1/2$. ✓

$$T(n) = \Theta(n^2)$$

Exemplo 7: $T(n) = 9T(n/3) + n$ — Caso 1



1. **Parâmetros:** $a = 9$, $b = 3$, $f(n) = n$.
2. **Folhas:** $n^{\log_3 9} = n^2$.
3. **Comparação:** $f(n) = n = O(n^{2-1})$. As folhas dominam.

$$T(n) = \Theta(n^2)$$

Exemplo 8: $T(n) = T(n/2) + 1$ — Caso 2 (Busca Binária)



1. **Parâmetros:** $a = 1$, $b = 2$, $f(n) = 1$.
2. **Folhas:** $n^{\log_2 1} = n^0 = 1$.
3. **Comparação:** $f(n) = 1 = \Theta(1)$. Empate.

$$T(n) = \Theta(\log n)$$

Exemplo 9: $T(n) = 3T(n/4) + n \log n$ — Caso 3

1. **Parâmetros:** $a = 3$, $b = 4$, $f(n) = n \log n$.
2. **Folhas:** $n^{\log_4 3} \approx n^{0.79}$.
3. **Comparação:** $f(n) = n \log n = \Omega(n^{0.79+\epsilon})$ para $\epsilon \approx 0.2$. A raiz domina.
4. **Regularidade:** $3 \cdot (n/4) \log(n/4) \leq \frac{3}{4} n \log n$, com $c = 3/4 < 1$. ✓

$$T(n) = \Theta(n \log n)$$

Exemplo 10: $T(n) = 7T(n/2) + n^2$ — Caso 1 (Strassen)



1. **Parâmetros:** $a = 7$, $b = 2$, $f(n) = n^2$.
2. **Folhas:** $n^{\log_2 7} \approx n^{2.81}$.
3. **Comparação:** $f(n) = n^2 = O(n^{2.81-0.81})$. As folhas dominam.

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$$

O algoritmo de Strassen reduz a de 8 para 7 (uma multiplicação a menos), melhorando a complexidade de $\Theta(n^3)$ para $\Theta(n^{2.81})$.

#	Recorrência	$n^{\log_b a}$	$f(n)$	Caso	Resultado
1	$4T(n/2) + n$	n^2	n	1	$\Theta(n^2)$
2	$2T(n/2) + n$	n	n	2	$\Theta(n \log n)$
3	$T(n/2) + n$	1	n	3	$\Theta(n)$
4	$8T(n/2) + n^2$	n^3	n^2	1	$\Theta(n^3)$
5	$4T(n/2) + n^2$	n^2	n^2	2	$\Theta(n^2 \log n)$
6	$2T(n/2) + n^2$	n	n^2	3	$\Theta(n^2)$
7	$9T(n/3) + n$	n^2	n	1	$\Theta(n^2)$
8	$T(n/2) + 1$	1	1	2	$\Theta(\log n)$
9	$3T(n/4) + n \log n$	$n^{0.79}$	$n \log n$	3	$\Theta(n \log n)$
10	$7T(n/2) + n^2$	$n^{2.81}$	n^2	1	$\Theta(n^{2.81})$

Síntese Final

Cheat Sheet: Guia Rápido de Complexidade

Iterativos — conte os laços:

- Sem laço $\Rightarrow O(1)$
- 1 laço sobre $n \Rightarrow O(n)$
- 2 laços aninhados $\Rightarrow O(n^2)$
- 3 laços aninhados $\Rightarrow O(n^3)$
- Divide espaço pela metade $\Rightarrow O(\log n)$

Recursivos — monte a recorrência:

- Identifique a (subproblemas), n/b ou $n-1$ (redução), $f(n)$ (combinar).
- Resolva via Árvore, Substituição ou Teorema Mestre.

Sinais de alerta:

- $2T(n-1) \Rightarrow O(2^n)$ — exponencial!
- $T(n-1) + \Theta(n) \Rightarrow O(n^2)$

O que Vem a Seguir

- **Próxima aula:** Exercícios práticos — análise de complexidade aplicada a algoritmos de ordenação e busca.
- **Leitura recomendada:** CLRS, capítulos 3 e 4 (Crescimento de Funções e Recorrências).

Pergunta para levar

O Quick Sort tem pior caso $O(n^2)$, mas na prática é mais rápido que o Merge Sort ($O(n \log n)$ garantido). **Por quê?** (Pense em constantes e no caso médio.)